

MySQL-Database-Engineering — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/MySQL-Database-Engineering`. Canonical sources stay in place.

Phase — 01-overview-and-scope

Directory: `01-overview-and-scope/`

Source: `01-overview-and-scope/01-path-goals-phase-map-and-mind-shift.md`

Path goals — moving from SQL syntax toward storage and systems impact

Goal: Stop treating MySQL usage as isolated string execution. Start reasoning about **how data is stored, how concurrency works, how plans cost resources, how replicas and backups behave, and how production constraints touch application design**.

Across this trace you practise on a **consistent ecommerce lab model** whenever a unit mentions it: users, products, categories, orders, order items, payments, addresses, reviews, coupons, inventory where relevant — only on databases and environments you fully control.

Study files are numbered for **topic order only** (not calendar scheduling unless you impose your own timetable).

Phase map (`NN-<area>/` — **worksheet counts** mirror your original phase depth; filenames `01` , `02` , ... are ordering only):

Area	Folder	Topics	Units
02	sql-fundamentals-query-thinking	Select / join / aggregate / UNION / EXISTS / CTE / windows / NULL / modeling / integration lab	10
03	index-architecture-and-explain-literacy	B-tree intuition, composites, cardinality, covering indexes, anti-patterns, <code>EXPLAIN</code> , large-shape drills	10
04	query-optimization-deepening	Plans, filesort, temp tables, joins, rewriter mindset, <code>EXPLAIN ANALYZE</code> , N+1, large data reality	10
05	transactions-and-concurrency	ACID, isolation, MVCC, locks, deadlock, races, transactional scope across checkout flows	10
06	storage-scaling-retention-archiving	Replication, lag, partitioning vs sharding narratives, backups, PITR, archiving, capacity sketches	10
07	high-availability-replication-topologies	Former “phase 5.2”: sync/async, topologies, failover/split brain, mirror vs backup, quorum intuition	6
08	production-database-operations	Slow log, pooling, infra signals, migrations/rollback discipline, restores, troubleshooting labs	10
09	interview-labs-and-scenarios	Drills → multi-service rehearsal + catalogue worksheet <code>09-*</code> (>50 checkpoints)	9
10	enterprise-boundaries-typescript-ssr-and-accessibility-for-data-products	TS at service boundaries, typed row/DTO contracts, narrow unions for state columns, composition over stringly typing, SSR/hydration when SQL feeds HTML, a11y for data-dense UIs	2

Completion bar (you define depth): Each phase ends when you can **explain why** typical choices behave the way they do on MySQL InnoDB workloads, using your own notebooks and artefacts — without treating interview lists as sterile memorisation.

Source: `01-overview-and-scope/02-overall-path-completion-mind-shift.md`

Overall completion criterion — path exit mindset

Shift from signalling “I know MySQL keywords” toward articulating how **relational storage, concurrency semantics, execution cost, replication and topology friction, operational controls, backup and readiness, retention posture, capacity growth,**

application versus database boundaries, and phased migration posture jointly shape **performance economics, reliability, scalability ceilings, cost pressure to the organisation, and blast radius containment when failures occur**.

Demonstrate competency through artefacts you authored (schemas, optimised statements, concurrency labs, HA sketches, rehearsed restores, metric correlation notes, handwritten trade-off briefs)—not through checklist completion alone.

Phase — 02-sql-fundamentals-query-thinking

Directory: 02-sql-fundamentals-query-thinking/

Source: 02-sql-fundamentals-query-thinking/01-from-data-to-information-select-basics.md

Unit 01 — From datum to information (relational chain)

Mindset: Database → relation → query → actionable information. `SELECT *` is seldom a deliberate design.

Concepts

Row selection, predicates, sorting, slicing, projecting only needed columns (`AS` , `LIMIT` , `DISTINCT`).

Build `users(id, email, country, created_at)` in your sandbox.

Practices

Recent signups sorted by creation time.

Filter by geography (your choice of predicate).

Countries list without duplicates (`DISTINCT`) with sensible ordering.

Avoid `SELECT *` in repeatable production-shaped queries except during one-off exploratory spelunking.

Interview angles

Difference between **filter semantics** (`WHERE`) and **result ordering** (`ORDER BY`): predicates vs presentation.

Why `LIMIT` interacts with `ORDER BY` (ordering must be deterministic when you slice).

When `DISTINCT` helps vs when it hides sloppy joins.

Source: 02-sql-fundamentals-query-thinking/02-join-thinking-inner-left-self.md

Unit 02 — Join thinking beyond single-table CRUD

Concepts

Inner vs left/right/self joins. Semantics when partner rows absent.

Keep using evolving ecommerce-shaped tables (`users` , `orders` , payments when present).

Deliver:

- Orders with owning user facts.
- Users who never placed orders (outer join pattern).
- Intentionally compare inner vs left for the same business question.

Lab

Articulate cardinality between result rows and predicates that move an inner join outcome toward empty sets.

Identify where **NULL-bearing columns signal optional relationships** versus bad data hygiene.

Interview angles

When `LEFT JOIN` encodes optional facts.

What inner join drops when no match exists.

NULL columns on the optional side vs inner join cardinality collapse.

Source: 02-sql-fundamentals-query-thinking/03-aggregation-reporting-group-having.md

Unit 03 — Aggregation and reporting primitives

Applications live on CRUD; leadership lives on aggregates.

Use `orders(amount, status, created_at)` (adapt names to your lab).

Skills: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, `HAVING`.

Practices

Daily revenue rollup.

Orders per geography or segment you model.

Higher-order customer spend statistics.

Top-N purchasers (defer window-function polish until the dedicated windows unit unless you peek ahead consciously).

Lab

Contrast `WHERE` (**pre-aggregation filter**) with `HAVING` (**post-aggregation predicate**) — write two variant queries illustrating each.

Enumerate common pitfalls: forgetting non-aggregated projections, grouping granularity mistakes.

Interview angles

Motivation for `HAVING` existence.

Mis-grouped aggregates and duplication inflation when joins widen grain before aggregates.

Source: 02-sql-fundamentals-query-thinking/04-union-and-union-all.md

Unit 04 — UNION vs UNION ALL semantics

Practice combining heterogeneous slices—for example partitioned yearly order tables (`orders_2025`, `orders_2026`) mocked in sandbox.

Goals:

- Harmonise column signatures.
- Decide when duplicate elimination (`UNION`) costs extra work vs `UNION ALL` fidelity.

Lab: Produce combined reporting totals and explain duplication handling strategy.

Interview: performance implication of `DISTINCT` pass inside `UNION`, and when uniqueness is logically guaranteed beforehand.

Source: 02-sql-fundamentals-query-thinking/05-subqueries-exists-and-in.md

Unit 05 — Subqueries, correlated forms, EXISTS / NOT EXISTS

Patterns:

Users without qualifying orders.

Catalogue items lacking purchase signals.

Ranking-style customer selections using subqueries (before leaning fully on windows).

Compare **semi-join intuition** (`EXISTS`) versus **membership checks** (`IN`) on shaped data where nullability lurks — document surprising tri-valued logic interplay.

Interview: correlated subquery amplification risks; when optimiser reshapes predicates (still reason from clarity first).

Source: 02-sql-fundamentals-query-thinking/06-cte-thinking-and-readability.md

Unit 06 — Common Table Expressions (CTE) for clarity

Stack multi-step derivations (e.g., `monthly_sales` CTE feeding `top_customers` CTE) inside a single statement.

Lab: Refactor a sprawling nested subquery into chained CTEs — comment on readability gains and when indirection still maps to similar plans (engine/version dependent; keep notes honest).

Interview tension: clarity vs perceived materialisation myths — treat CTE as organisational first, measure second when performance matters.

Source: 02-sql-fundamentals-query-thinking/07-window-functions-partitions-and-ranking.md

Unit 07 — Window functions (ranking and running analytics)

Functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, framing with `OVER (PARTITION BY ... ORDER BY ...)`.

Deliver:

- Top purchasers per month buckets.
- Product leaderboards with tie semantics called out.
- Running totals / moving metrics on order streams you generate.

Lab articulate **aggregate collapse** (`GROUP BY`) vs **per-row analytic** (`OVER`): when each mental model fits.

Interview: windows for ranking & cumulative metrics; aggregates for dataset compression.

Source: 02-sql-fundamentals-query-thinking/08-null-behavior-three-valued-logic.md

Unit 08 — NULL semantics and defensive expressions

Use payment-like columns (`paid_at`, nullable `coupon_id`, `refund_reason`) to observe sentinel vs unknown distinctions.

Practice `COALESCE`, `NULLIF`, MySQL-specific `IFNULL` awareness (portability caveat).

Labs:

Explain why `NULL = NULL` is not true in SQL logic.

Contrast `COUNT(*)` grain vs `COUNT(column)` omission of NULL rows.

Interview: propagation of UNKNOWN in predicates; unintended outer-join fallout.

Source: 02-sql-fundamentals-query-thinking/09-relational-modeling-keys-and-normalization.md

Unit 09 — Relational modelling guardrails

Revisit ecommerce entities with explicit **primary keys**, **foreign keys**, **uniques**, normalization targets, ownership boundaries (who mutates what), cardinalities (`1:N`, `M:N` via junction).

Lab: Sketch a deficient schema (duplication / fan traps) then remediate documenting relationship narratives.

Interview: cardinality patterns, bridging tables, surrogate vs natural keys trade-offs (pragmatic framing, not ideology).

Source: 02-sql-fundamentals-query-thinking/10-integration-ecommerce-dashboard-and-analytics.md

Unit 10 — Phase integration lab (reports + dashboards + analytics breadth)

Produce a coherent schema crossing users, products, orders, payments, reviews, coupons (subset acceptable if staged).

Deliver SQL artefacts:

Operational slice: users lacking orders.

Commercial slice: ranked customers.

Temporal revenue reporting.

Ranked catalogue performance narratives.

Capture **explicit trade-offs** per query (grain, cardinality risk, readability vs reuse).

Exit criterion: Explain how table relationships—not clever syntax alone—determine viability of answers.

Phase — 03-index-architecture-and-explain-literacy

Directory: 03-index-architecture-and-explain-literacy/

Source: 03-index-architecture-and-explain-literacy/01-scan-vs-seek-index-intuition.md

Unit 01 — Index as structure, not superstition

Contrast full scan vs indexed lookup on `orders(id, customer_id, status, created_at)` shaped lab rows.

Observe predicate `WHERE customer_id = ?` performance before vs after auxiliary index introduction.

Articulate logarithmic retrieval intuition rooted in balanced tree ordered keys (conceptual drawing welcome).

Interview: coarse B-tree mental model versus linear scan asymptotics at scale narratives.

Source: 03-index-architecture-and-explain-literacy/02-b-tree-range-and-non-sargable-patterns.md

Unit 02 — B-tree mental model: ordering, ranges, sabotaged predicates

Index on `created_at`; query rolling windows (`created_at >= ...`).

Highlight why leading wildcards (`LIKE '%token%'`) sabotage typical B-tree seek usage.

Lab: predict plan shape, confirm with `EXPLAIN` (version-specific column naming—note your server).

Interview: complexity comparison scan vs seek; selective range vs ambiguous pattern matching.

Source: 03-index-architecture-and-explain-literacy/03-composite-index-column-order-experiments.md

Unit 03 — Composite index column ordering

Contrast `(customer_id, status)` versus `(status, customer_id)` under realistic compound predicates (`customer_id=? AND status=?` vs status-only probes).

Hypothesise mismatches vs actual optimiser selections.

Interview: coupling selectivity anecdotes with cardinality awareness (preview next units).

Source: 03-index-architecture-and-explain-literacy/04-leftmost-prefix-rule-drills.md

Unit 04 — Leftmost prefix rule interrogation

Index `(country, status, created_at)` — derive which predicates can leverage prefix segments.

Craft ~10 exploratory queries forecasting usable segments vs residual scans — verify.

Interview recap: reordering composites without measuring access patterns wastes opportunity.

Source: 03-index-architecture-and-explain-literacy/05-cardinality-selectivity-low-signal-columns.md

Unit 05 — Cardinality / selectivity: not every column earns an index

Build intentionally low-distinct indices (`status` tri-state) contrasting high-uniqueness (`email`-like).

Observe optimiser abandonment or downgrade where histograms signal poor payoff.

Interview: describing low cardinality index traps and mitigation (composite leading strong key, constraint redesign, conditional indexes if platform supports—note portability).

Source: 03-index-architecture-and-explain-literacy/06-covering-index-and-narrow-projections.md

Unit 06 — Covering indexes and index-only access paths

Benchmark `SELECT customer_id, status FROM orders WHERE customer_id=?` leveraging composite aligning projection with index columns.

Contrast widened projections forcing heap/table lookups (`SELECT *`).

Interview articulate covering index payoff vs write amplification burdens.

Source: 03-index-architecture-and-explain-literacy/07-explain-thinking-and-access-types.md

Unit 07 — EXPLAIN literacy (access types sanity)

Iterate ~20 candidate queries tagging `ALL`, `range`, `ref`, `const`, residual `filtered` guesses.

Diagnose ominous breadth (`ALL`), suspicious row estimates versus actuals when `EXPLAIN ANALYZE` available.

Interview clarify meaning of `rows` (estimate historically) versus measured timings.

Source: 03-index-architecture-and-explain-literacy/08-query-anti-pattern-repair-set.md

Unit 08 — Operational anti-pattern catalogue remediation

Practice repairing:

Broad star projections.

Wildcard middle patterns.

Datetime functions sabotaging predicates (`WHERE YEAR(created_at)=...` rewritten to sane ranges).

N+1 application shapes (conceptual synergy with Phase 04 deeper unit).

Shoot for ~15 rewrites annotated with rationale.

Interview link: predicate sargability, index alignment.

Source: 03-index-architecture-and-explain-literacy/09-deliberately-slow-query-lab.md

Unit 09 — Slow query lab (intentional harm)

Produce pathological combinations: absent index, mis-ordered composites, malignant `LIKE`, large grouping without supportive ordering keys.

Iterate `EXPLAIN` → hypothesis → corrective index/query adjustments → annotated journal.

Source: 03-index-architecture-and-explain-literacy/10-integration-large-shape-index-strategy.md

Unit 10 — Integration lab (analytics at stress volume)

Assume or simulate large cardinality customers (sandbox scale you can ethically generate): millions of accounts, exaggerated orders.

Author analytics / dashboards / KPI queries tying back to purposeful composite + covering selections.

Explain **why** each primary index serves the dashboards—not merely listing artefacts.

Phase — 04-query-optimization-deepening

Directory: 04-query-optimization-deepening/

Source: 04-query-optimization-deepening/01-execution-plan-structure.md

Unit 01 — Execution plan as contract with the optimiser

Map EXPLAIN fields (id, select_type, table, type, rows, key, Extra) to mental story of join order and access path.

Predict then verify on orders filtered by customer_id.

Interview: interpret rows estimate; interpret type degradation.

Source: 04-query-optimization-deepening/02-access-type-deep-dive.md

Unit 02 — Access type continuum (prefer tight probes)

Understand favorable classes (const, eq_ref, ref, range) vs broader (index, ALL).

Build paired good/bad predicates demonstrating transitions.

Interview: rationale that ALL is not universally evil but alarming at scale without compensating tactics.

Source: 04-query-optimization-deepening/03-filesort-and-order-by-cost.md

Unit 03 — Sorting spill: Using filesort

Detect ORDER BY without aligning index ordering at large cardinality.

Iterate micro-benchmark introducing supportive composite index aligning sort keys.

Interview: articulate cost of external sort versus streaming index order.

Source: 04-query-optimization-deepening/04-temporary-table-and-group-by-pressure.md

Unit 04 — Temporary tables surfaced in plans

Observe GROUP BY operations elevating temporary storage or memory pressure (Using temporary in legacy EXPLAIN extras —align wording to your version docs).

Stress large fan-in grouping; hypothesise optimisation levers.

Interview: interplay of aggregate algorithms and memory thresholds (high-level honesty if exact engine thresholds unmemorised —look up authoritative docs when precision needed).

Source: 04-query-optimization-deepening/05-join-strategy-nested-loop-and-driving-table.md

Unit 05 — Join strategy narration

Walk multi-table joins across users / orders / payments variants.

Discuss nested-loop mental model on InnoDB workloads, notion of driving table hypothesis, cardinality steering order.

Interview: pragmatic contrast with hash joins if engine introduces them later—stay version aware.

Source: 04-query-optimization-deepening/06-optimizer-behaviour-and-cost-blindspots.md

Unit 06 — Optimiser autonomy (and occasional miscues)

Manipulate purposely misleading secondary indexes prompting ignore or alternate path selection.

Document cost model dependencies: stale stats (run `ANALYZE TABLE` ethically in sandbox), skew, histogram limitations per version.

Interview: caution on hint reliance; favour measurement + clarity of intent.

Source: 04-query-optimization-deepening/07-query-rewrite-equivalence-with-lower-cost.md

Unit 07 — Equivalence-preserving rewrites

Transform ~20 statements:

Narrow projections.

Range-friendly datetime predicates.

Revisit `IN` vs `EXISTS` story with measured evidence on your dataset.

Interview: rewrite discipline—never assume identical plans post cosmetic change.

Source: 04-query-optimization-deepening/08-n-plus-one-and-batch-loading-discipline.md

Unit 08 — N+1 pathology and batching discipline

Simulate ORM-style repeated single-row fetch sequences (Doctrine-like mental model) versus join or batched strategies.

Measure round trips / statement counts (even crudely).

Interview: eager vs lazy loading trade-offs; API layer fan-out responsibility.

Source: 04-query-optimization-deepening/09-large-dataset-stress-testing.md

Unit 09 — Large dataset truth-telling

Generate or import tens of millions of rows only where your local resources and time allow; otherwise scale sublinearly with honest extrapolation caveats in your notes.

Stress `GROUP BY`, `JOIN`, and `ORDER BY`; document mitigations and observed plan shifts.

Source: 04-query-optimization-deepening/10-integration-dashboard-at-scale.md

Unit 10 — Integration lab (dashboard KPIs on wide fact tables)

Assume large user and order populations; define dashboard metrics: revenue aggregates, top cohorts, growth counters, refund rates.

For each metric set: index rationale, join choreography, plan snapshot, cost narrative.

Interview synthesis: `EXISTS` vs `IN`, join vs subquery, filesort, temps, N+1 avoidance, optimiser surprises, star projection penalties, full scans when acceptable.

Phase — 05-transactions-and-concurrency

Directory: 05-transactions-and-concurrency/

Source: 05-transactions-and-concurrency/01-acid-motivation-and-boundaries.md

Unit 01 — ACID framing for multi-step business mutations

Script inventory decrement + payment acknowledgement inside transaction boundaries; inject secondary statement failure to observe atomic rollback necessity.

Interview: durability expectations vs fsync / replication semantics (stay honest about depth).

Source: 05-transactions-and-concurrency/02-isolation-levels-and-phenomena.md

Unit 02 — Isolation level ladder (conceptual reproduction)

Compare `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE` narratives.

MySQL / InnoDB default typically `REPEATABLE READ` — validate on your instance (`SHOW VARIABLES LIKE 'transaction_isolation';`).

Labs attempt dirty read / non-repeatable / phantom class demonstrations where engine semantics allow (some phenomena intentionally prevented—document outcomes truthfully).

Source: 05-transactions-and-concurrency/03-mvcc-snapshot-reads.md

Unit 03 — MVCC snapshots

Dual-session read/write interplay illustrating consistent reads without necessarily blocking writers (nuanced with locking reads later).

Link undo log / version chain intuition at high level.

Interview: why plain `SELECT` often avoids classic X-lock mutual exclusion compared to lock-in-mode reads.

Source: 05-transactions-and-concurrency/04-locking-row-gap-next-key.md

Unit 04 — Lock vocabulary (row, gap, next-key intention)

Exercise `SELECT ... FOR UPDATE` producing blocking second writer updates.

Map gap / next-key locking purpose for phantom protection in RR (engine-specific—cite references you consult).

Interview: describe waiting vs deadlock distinction at conceptual layer.

Source: 05-transactions-and-concurrency/05-pessimistic-locking-for-update-flow.md

Unit 05 — Pessimistic locking in checkout flows

Serialise inventory mutation using `FOR UPDATE` path; measure contention.

Interview: pessimistic applicability when collisions frequent or business tolerates fewer optimistic retries.

Source: 05-transactions-and-concurrency/06-optimistic-locking-version-columns.md

Unit 06 — Optimistic locking via version counters

Add `version INT` pattern; craft concurrent updates violating version predicate to surface application-level retry story.
Contrast with pessimistic friction under high churn.

Source: 05-transactions-and-concurrency/07-deadlock-reproduction-and-remediation.md

Unit 07 — Deadlock choreography and mitigation

Two sessions acquire locks in opposing order on `orders` vs `inventory`; force engine deadlock detection.
Discuss prevention: consistent lock acquisition ordering, smaller critical sections, backoff / retry strategy at app layer.

Source: 05-transactions-and-concurrency/08-race-condition-lost-updates.md

Unit 08 — Race conditions without proper atomicity

Simulate read–modify–write lost update on inventory counters under concurrent sessions; repair with single-statement atomic decrement or transactional discipline.
Interview: compare database constraint vs application orchestration responsibilities.

Source: 05-transactions-and-concurrency/09-transaction-scope-and-contention.md

Unit 09 — Transaction scope discipline

Anti-pattern: elongated transactions bundling network I/O, external payment APIs, analytics side effects.
Good pattern: commit hot data mutations quickly; offload slow work post-commit with idempotent follow-ups.
Lab: refactor narrative transaction boundaries (even paper design if code not ready).

Source: 05-transactions-and-concurrency/10-integration-concurrency-incident-lab.md

Unit 10 — Integration lab (scale + failure classes)

Model large order history; inject deliberate deadlock scenarios, race regressions, locking hotspots.
Document detection (`SHOW ENGINE INNODB STATUS` historical patterns, modern performance schema usage per version) and corrective stories.
Interview recap: ACID, MVCC, isolation defaults on MySQL, phantom/repeatable read framing, pessimistic vs optimistic, deadlock strategy.

Phase — 06-storage-scaling-retention-archiving

Directory: `06-storage-scaling-retention-archiving/`

Source: `06-storage-scaling-retention-archiving/01-replication-motivation-ha-vs-read-scale.md`

Unit 01 — Replication purpose map

Sketch primary servicing writes, replicas extending read scalability / DR orientation (distinct goals).

Articulate Symphony-style app traffic routing hypothetical (writes vs analytics reads).

Interview: misconception that replicas are magically consistent read pools.

Source: `06-storage-scaling-retention-archiving/02-binary-log-and-replication-modes.md`

Unit 02 — Binary log modalities (conceptual contrast)

Discuss row vs statement replication trade-offs (verbosity, nondeterministic statement hazards, portability).

Produce pros/cons bullets referencing official docs for authoritative nuance—avoid memorised absolutes.

Source: `06-storage-scaling-retention-archiving/03-replica-lag-read-after-write-framing.md`

Unit 03 — Replica lag and read-your-writes pitfalls

Tell story: payment success on primary followed by immediate stale read replica missing row.

Classify workloads tolerating eventual consistency versus those requiring staleness shields (routing critical reads back to primary, feature-level fallbacks—not hand-wavy “just use caching” defaults).

Source: `06-storage-scaling-retention-archiving/04-read-write-splitting-patterns.md`

Unit 04 — Read/write splitting patterns

Model order placement vs reporting dashboards with explicit topology statements.

Enumerate trade-offs: credential complexity, connection fan-out, staleness disclosures to product owners.

Source: `06-storage-scaling-retention-archiving/05-partitioning-range-and-hash.md`

Unit 05 — Partitioning strategies (native table partitions)

Use large log-style tables (`audit_logs`) to reason about RANGE / HASH partition sketches.

Articulate scenarios where partitioning adds operational burden without payoff (tiny tables, uneven hot partitions).

Interview compare **partition vs shard** succinctly—scope of coupling, routing, cross-shard joins.

Source: `06-storage-scaling-retention-archiving/06-sharding-as-last-resort.md`

Unit 06 — Sharding last-resort calculus

Enumerate routing complexities, migration pain, transactional boundary erosion, observability fragmentation.

Produce a written decision to defer horizontal sharding until vertical optimisations, archiving, partitioning, and read-scale paths are exhausted.

Source: `06-storage-scaling-retention-archiving/07-backup-architecture-and-pitr-framing.md`

Unit 07 — Backups plus point-in-time recovery narrative

Articulate differential layers: logical vs physical snapshots, incremental strategies, binlog chaining for replay windows.

Practice-only exercise (authorised sandbox): simulate destructive deletes on disposable schema followed by scripted restore—you document exact procedure references you followed.

Interview: untreated backup mythology (“we have nightly mysqldump therefore safe”) versus validated restore rehearsals.

Source: `06-storage-scaling-retention-archiving/08-archiving-retention-and-deletion-tradeoffs.md`

Unit 08 — Retention policies, archiving paths, lawful deletion semantics

Contrast cold archive tables / object storage offloads versus hard deletes with auditing constraints.

Produce policy bullets for orders vs logs vs audit trails balancing compliance unknowns—you flag legal review externally when needed.

Source: `06-storage-scaling-retention-archiving/09-storage-growth-modelling-capacity-thinking.md`

Unit 09 — Storage growth modelling worksheet

Assume payload averages, churn, multi-year horizons—approximate cumulative bytes and annual growth deltas.

Discuss index amplification (secondary indexes occupy space and write bandwidth too).

Source: `06-storage-scaling-retention-archiving/10-integration-large-audit-retention-drill.md`

Unit 10 — Integration lab (users, massive orders, huge audit posture)

Architect primary + replicas + backups + retention + partition/archive strategy narration.

Stress-test failure stories: replication delay emergencies, brittle backup durations, exploding disk utilisation—you outline mitigations succinctly.

Interview recap cluster: replication lag reasoning, shard vs partition, PITR, HA vs backup distinctions, replication mode trade-offs.

Phase — 07-high-availability-replication-topologies

Directory: `07-high-availability-replication-topologies/`

Source: `07-high-availability-replication-topologies/01-async-sync-and-semi-sync-replication.md`

Unit 01 — Async vs synchronous replication trade-offs

Articulate latency vs durability tension; semi-sync compromises.

Interview: scenarios where synchronous replication destroys user-visible latency budgets.

Source: `07-high-availability-replication-topologies/02-replication-topology-patterns.md`

Unit 02 — Topology sketches (chains, multiples, contested multi-primary)

Draw primary → replica fans, chaining, cautions about cascading lag amplification.

Enumerate multi-primary complexity without diving vendor-specific quorum details yet.

Source: `07-high-availability-replication-topologies/03-failover-split-brain-and-orchestration-awareness.md`

Unit 03 — Failover risk and split-brain metaphors

Narrate dual-primary belief scenarios absent fencing—high-level orchestration primitives (witness, quorum) remain literature follow-ups unless you operate them hands-on elsewhere.

Interview: automated failover bravery vs staged human checkpoints.

Source: `07-high-availability-replication-topologies/04-mirror-replica-vs-backup-distinctions.md`

Unit 04 — Mirrors are not backups

Demonstrate logically destructive propagation through replication contrasts with time-delayed/offline snapshots.

Interview: succinct differentiation mirror / replica / backup role boundaries.

Source: `07-high-availability-replication-topologies/05-quorum-consensus-raft-awareness.md`

Unit 05 — Consensus intuition (architecture depth cap)

Expose yourself to Raft / quorum primers at conceptual layer only—tie to why HA stacks outside raw MySQL exist.

Explicitly fence scope: avoid claiming DBA-specialist competency from surface reading alone.

Source: `07-high-availability-replication-topologies/06-integration-ha-architecture-documentation-lab.md`

Unit 06 — HA integration lab narration

Produce an architecture diagram or structured narrative: millions of accounts, resilient primary/replica stance, failover expectations, backups + PITR overlay, chosen replication modality justification.

Enumerate residual operational risks plainly for hypothetical stakeholders.

Phase — 08-production-database-operations

Directory: 08-production-database-operations/

Source: 08-production-database-operations/01-production-triage-thinking.md

Unit 01 — Production mindset diagnostics ordering

Enumerate triage playbook: classify symptom → corroborating metrics → likely classes (CPU saturation vs IO vs locking vs backlog) rather than blindly vertical scaling reflex.

Sandbox inject slow query spike plus synthetic connection pressure if ethically possible on disposable instances.

Source: 08-production-database-operations/02-slow-query-log-practice.md

Unit 02 — Slow query log instrumentation

Exercise enabling / tuning slow query thresholds (sandbox). Correlate surfaced statements with remediation backlog.

Interview: pitfalls of excessively low thresholds drowning signal.

Source: 08-production-database-operations/03-connection-pools-and-exhaustion-attacks.md

Unit 03 — Connection management and pool exhaustion narratives

Discuss `max_connections`, pool sizing interplay with PHP-FPM or other worker concurrency models, lingering idle leases.

Produce a hypothetical incident brief: saturation without visibly slow SQL (connection storms / pool misconfiguration).

Source: 08-production-database-operations/04-resource-bottleneck-triangle-cpu-ram-disk.md

Unit 04 — CPU, memory, disk pressure correlations

Associate CPU hotspots with hashing/sorting, RAM with buffer/cache pressure thresholds, disk with IOPS + latency envelope breaches.

Observe heavy aggregate queries elevating measurable signals deliberately in lab.

Source: 08-production-database-operations/05-migration-expand-contract-zero-downtime-awareness.md

Unit 05 — Evolutionary schema migration posture

Articulate additive backward-compatible rollout patterns (expand phase, dual-write/read bridging, eventual contract tightening) referencing industry literature.

Avoid naïvely blocking `ALTER` on hypothetical 150M-row core fact table without phased plan—even if simulated only intellectually until you practise on clones.

Source: 08-production-database-operations/06-rollback-and-forward-fix-discipline.md

Unit 06 — Deploy rollback interplay with database state

Case study: flawed column rollout—decide reversible application paths vs compensating forwards-only DB fixes.

Source: 08-production-database-operations/07-restore-validation-rehearsals.md

Unit 07 — Restoration proof exercises

Treat backup artefacts as unvalidated until exercised through full restore rehearsal onto disposable host.

Articulate discrepancy detection steps post-restore checksum spot checks where applicable.

Source: 08-production-database-operations/08-retention-jobs-operational-awareness.md

Unit 08 — Retention enforcement operations

Operationalise TTL-style jobs, archiving batch windows, and anomaly detection hooks when pruning silently stalls—tie each control to stakeholder-visible risk narratives.

Source: 08-production-database-operations/09-composite-incident-troubleshooting-lab.md

Unit 09 — Troubleshooting composite incident lab

Synthetic multi-signal outages: soaring CPU alongside replica lag vs lock waits vs saturation—produce ordered investigation notes.

Close each thread with corrective hypothesis + validation criterion.

Source: 08-production-database-operations/10-integration-production-incident-response-lab.md

Unit 10 — Final production integration scenario

Stakeholder brief: degraded dashboard, CPU saturation, lengthening backups, creeping disk utilisation concurrent with intermittent replica staleness—you outline prioritised investigative sequence, containment posture, systemic fixes versus tactical patches.

Explicitly weave rollback narratives and validated restore rehearsals when contemplating schema mutation under fire.

Phase — 09-interview-labs-and-scenarios

Directory: 09-interview-labs-and-scenarios/

Source: 09-interview-labs-and-scenarios/01-query-optimization-interrogation-drills.md

Unit 01 — Query optimisation interrogation drills

Given pathological predicates (functions on indexed timestamps, malignant `LIKE`, star projections)—produce rewritten SQL, justification sketch, hypothetical index deltas, residual risk disclosures.

Maintain a personally authored bank of optimisation micro-puzzles (target ≥ 20 over time).

Source: 09-interview-labs-and-scenarios/02-deadlock-concurrency-storytelling-interview.md

Unit 02 — Deadlock reproduction oral defence

Articulate chronological lock acquisition storyline for crossed updates across inventory versus orders domains; remediation via deterministic lock ordering conventions, backoff or retry choreography, narrower transactional footprints.

Source: 09-interview-labs-and-scenarios/03-whiteboard-ecommerce-schema.md

Unit 03 — Whiteboard ecommerce schema narration

Produce schema covering users, products, inventory pivots, orders, coupons, audits—annotate PK/FK patterns, cardinality, eventual retention hooks.

Source: 09-interview-labs-and-scenarios/04-scaling-tradeoff-scenario-drill.md

Unit 04 — Scaling decision scenario

Assume millions of monthly active purchasers and tens of millions of historic orders atop sluggish dashboards—articulate investigative branches: replica amplification, archiving, partitioning, pessimistic avoidance of premature sharding—each with drawback disclosure.

Source: 09-interview-labs-and-scenarios/05-incident-hot-cpu-disk-and-throughput-collapse.md

Unit 05 — Production incident triage under vague symptoms

Case: dashboards timing out despite orders path apparently healthy—investigation ordering (slow logs, InnoDB monitors, filesystem pressure, saturation of replicas, rogue batch jobs)—you script talking points aloud.

Source: 09-interview-labs-and-scenarios/06-large-order-table-migration-scenario.md

Unit 06 — Heavy table migration interrogation (wide `orders` fact table adding `country`)

Plan additive nullable column phased adoption, backwards compatible deploy choreography, reversible steps, observable risk telemetry.

Prep answers anchored in expand/contract sequencing literature you intentionally study—not improvisational folklore.

Source: 09-interview-labs-and-scenarios/07-large-architecture-narrative-lab.md

Unit 07 — Comprehensive architecture narration lab

Across users, transactional orders, downstream analytics aggregates, sprawling audit artefacts—diagram replication stance, thoughtfully scoped partitioning, archiving plus PITR posture, interplay with ancillary services—all justified via **why** paragraphs instead of acronym soup inventory dumps.

Source: 09-interview-labs-and-scenarios/08-multi-service-pressure-cooker-lab.md

Unit 08 — Full-stack pressure rehearsal (timed architect drill)

Synthetic stack narrative: Symphony API → MySQL → Redis auxiliaries → asynchronous Go-flavoured workers—collectively alleged to buckle under accelerating order ingestion with CPU hotspots, lethargic backup windows, SLA-breaching dashboards, measurable replica divergence.

Spend roughly ninety self-timed minutes:

1. List clarifying questions you would escalate to ambiguous stakeholders/product owners/SRE counterparts.
 2. Enumerate prioritised hypotheses with observable signals capable of falsifying each.
 3. Outline containment versus structural remediation arcs with sequencing honesty.
 4. Surface trade-offs (financial, organisational, correctness, staleness allowances) without collapsing into techno-optimistic hand-waving.
-

Source: 09-interview-labs-and-scenarios/09-interview-question-catalog.md

Unit 09 — Condensed interview prompt catalogue (>50 checkpoints)

Use this catalogue for orientation—not rote cramming. Expand bullets with artefacts from labs.

SQL semantics

1. Explain `WHERE` versus `ORDER BY`.
2. `LIMIT` interplay with nondeterministic ordering risks.
3. `DISTINCT` cost intuition.
4. Inner vs outer join fallout on unmatched rows.
5. `NULL`-bearing columns meaning under outer joins.
6. Aggregate pitfalls (implicit grouping grain / forgotten keys).
7. `HAVING` purpose versus mis-filtering aggregates with `WHERE` alone.
8. `UNION` duplicate elimination implication.
9. `UNION ALL` when uniqueness is guaranteed upstream.
10. Correlated vs uncorrelated subquery clarity.
11. `EXISTS` vs `IN` with `NULL` pitfalls.
12. CTE readability vs measure-after refactor discipline.
13. Window partition grain vs collapsing aggregates (`GROUP BY`).
14. `ROW_NUMBER` vs `RANK` tie semantics.
15. Running totals via windows versus awkward self joins.
16. Three-valued logic and `NULL` predicates.
17. `COUNT(*)` vs `COUNT(column)` omissions.
18. Join fan-out exploding aggregates before collapsing.

Indexing literacy

1. B-tree ordered key intuition.

2. Leftmost composite prefix rules.
3. Low-cardinality lone secondary index traps.
4. Covering index vs write-amplification converse.
5. Selectivity ties to cardinality.
6. Explain extras such as covering / filter pushdown cues (consult your engine version docs for exact wording).
7. Leading-wildcard `LIKE` and index abandonment.
8. Functions on indexed columns → range-friendly rewrites.
9. Index-only access vs widening projections.
10. Composite column order mismatch with real query predicates.
11. Stale optimizer statistics contributing to regressions (`ANALYZE TABLE` awareness).

Optimisation narratives

1. Interpret `type` tending toward full scans.
2. Trust boundaries for estimated `rows`.
3. Filesort avoidance via aligned sort keys / indexes.
4. Temporary materialisation signals around heavy aggregates.
5. Nested-loop join mental model (engine evolution caveat).
6. Driving-table heuristics vs measurement.
7. Optimiser declining a seemingly usable index—skew and costing story.
8. Semantics-preserving rewrites lowering cost.
9. `SELECT *` widening payloads.
10. Datetime predicates as half-open ranges instead of wrapping functions.

Through-application patterns

1. Semi-join rewrites (`IN` vs `EXISTS`) measured, not dogmatic.
2. N+1 symptom patterns across ORMs (e.g., Doctrine analogue).
3. Batch loading / join consolidation remediation.
4. Small-row-count lies versus large-shape truth.
5. Oversized transactions and contention.
6. Holding transactions across network / external APIs anti-pattern.

Concurrency lore

1. ACID with concrete failure stories—not acronyms alone.
2. Isolation ladder phenomena you personally attempted to illustrate (honest reproducibility caveats).
3. MVCC snapshot read intuition.
4. Gap / next-key locking purpose tied to repeatable-read phantom mitigation (cite official docs layers you used).
5. Pessimistic `FOR UPDATE`.
6. Optimistic versioning collisions.
7. Deadlock cyclic acquisition remediation.
8. Lost-update races absent atomic decrement / proper isolation layering.
9. InnoDB isolation default reproducible statement—verify with `SHOW VARIABLES` on **your** instance.

Replication, scaling, DR

1. Replication for read scale vs failover vs HA objectives—do not conflate blindly.
2. Async replication staleness impacting read-after-write expectations.
3. Replica lag alerting versus feature-level staleness shields.
4. Shard vs partition differentiator—scope of routing and cross-boundary joins complexity.
5. Sharding complication inventory (routing, resharding migrations, ops fragmentation).
6. Mirrors / replicas versus offline backups.
7. PITR story: coordinated snapshot + redo / binlog chain (follow vendor docs wording).
8. Capacity sketch including index amplification and retention deltas.

Production operations

1. Slow log threshold producing noise paralysis.
 2. Connection pool starvation distinct from purely slow SQL.
 3. Correlate CPU / memory / disk / IOPS narratives with plausible DB internals.
 4. Incident triage ordering under simultaneous anomalies (dashboards + saturation + replicas).
 5. Retention pruning job silent failure swelling storage.
 6. Verified restore rehearsals versus checkbox backups.
 7. Expand/contract migration talking points anchored in literature you actually read.
 8. Rollforward-only correction strategies when DDL cannot roll back cleanly.
-

Phase — 10-enterprise-boundaries-typescript-ssr-and-accessibility-for-data-products

Directory: 10-enterprise-boundaries-typescript-ssr-and-accessibility-for-data-products/

Source:

10-enterprise-boundaries-typescript-ssr-and-accessibility-for-data-products/01-scope-typescript-contracts-ssr-hydration-accessibility-for-database-backed-products.md

Unit 1 — Scope: enterprise-facing boundaries around MySQL-backed products

Pure SQL literacy is not enough when data **leaves InnoDB** — it becomes JSON over HTTP, SSR HTML, or client state. This area tightens the **contract layer** so schema drift, ambiguous unions, inaccessible admin UIs, and hydration bugs do not undo diligence from earlier **MySQL worksheets** here (indexing 03-* through 09-* interview blocks).

Learning outcomes

TypeScript + MySQL row shapes (application discipline)

- Separate **row type** (exactly what `SELECT` returned) from **domain type** (what the business module may assume after validation).
- Generics for **repositories**: `Repository<TRow, TId>` with constraints `TRow extends { id: TId }` — articulate when generics buy safety vs ceremony.
- **Union + narrowing** for nullable columns, status `ENUM`s, soft-delete flags — never leave `string` as the dumping ground where a finite domain exists.
- Prefer **narrowing via validation** (`zod`/`valibot`/`typeguard`) at process boundaries—not blind casts from `mysql2` row arrays.

Composition patterns crossing the DB boundary

- Build **pipes**: `mysql row → validated DTO → view model`; document where immutability and `readonly` help concurrent refactors across services.
- Avoid “god typings” that paste entire tables into SPA bundles—surface **minimal projections** typed explicitly.

SSR & hydration — conceptual mastery for architects

- Explain **SSR** when dashboards or marketing surfaces must ship HTML with authoritative numbers from SQL.
- Understand **hydration**: client attaches behaviour to server-rendered markup; articulate root causes when IDs, timestamps, locale formatting, randomised marketing copy, or A/B flags diverge server vs browser.
- Name mitigations beyond `suppressHydrationWarning`: stabilise entropy sources, unify clock/locale negotiation, consolidate feature flags upstream.

Accessibility for data-heavy UIs

- Heading hierarchy when tables represent nested entities.
- `scope` **semantics** vs ARIA-only shortcuts for sortable columns.
- Keyboard focus when inline editing cells or running dangerous SQL-adjacent actions (delete, refund) from admin panels.
- Focus management when modals confirm destructive operations.

Enterprise interview articulation

You should whiteboard how **schema change** (new nullable column) propagates through **types**, **API version**, **SSR payload**, and **client cache** without hand-wavy “we’ll fix later”.

Positioning inside this path

This is **not** a replacement for a dedicated frontend track — it is the **interface consciousness** every storage engineer working next to TypeScript services or Vue apps must own. Pair with `paths/JavaScript-and-Vue/03-*` if you need front toolkit depth; here the lens stays **data fidelity + consumer UX risk**.

Deliverable mindset

Any exercise you finish should answer: **If the column lies, which layer catches it, and what does the user experience before engineering notices?**

Source: `10-enterprise-boundaries-typescript-ssr-and-accessibility-for-data-products/02-labs-typed-boundaries-and-consumer-ux-tabletops.md`

Unit 2 — Labs: typed seams + UX failure tabletops

Operate on a schema you already practise (ecommerce lab or smaller inventory subset). Use **safe** local/dev data only.

Lab 1 — Generic repository boundary (TypeScript)

Implement a **narrow** repository interface for one aggregate (e.g. `Order`):

- `findById(id: BrandedOrderId): Promise<OrderRow | undefined>` using **branded IDs** (string literal brand) to stop cross-pollination with `CustomerId`.
- `listForUser(userId: UserId, page: PageCursor): Promise<OrderSummaryDto[]>` returning **DTO projections** (fewer columns than raw row type).
- Annotate where `readonly` on returned arrays prevents accidental in-place mutation leaking to UI stores.

Document one scenario where **GENERIC** default parameters would collapse inference if you mishandle `const` parameters.

Lab 2 — Union + narrowing for order pipeline

Model `order_status` transitions as a **finite union** (`placed` → `paid` → `fulfilled` ... + `cancelled` with reason). Build:

- Parser from **stringly** DB `ENUM` / `VARCHAR` column into the union.
 - `assertNever` guarded reducer printing human-readable audit text.
 - Unit tests showing illegal transitions **fail at type level** once you encode state machine as types (lightweight version acceptable).
-

Lab 3 — Composition of DTO layers (no mega-type export)

Compose types using `Pick`, `Omit`, and intersection:

- `PaymentRow` (wide)
- `PaymentPublicDto` for external API responses
- `PaymentAdminDto` exposing operator metadata

Explain **why** shipping `PaymentRow` verbatim to SPA bundles is hostile to security posture + bundle size—even if HTTPS exists.

Lab 4 — SSR / hydration narrative (desk exercise)

Assume an internal dashboard SSRs KPI numbers straight from nightly SQL rollup.

Write a memo covering:

1. Sources of divergence between SSR HTML totals and SPA-hydrated chart components.
2. How **replica lag** interacts with SSR if read traffic hits async replica pools.
3. Rollback playbook if hydrated UI throws—what happens to stale interactive shell?

Lab 5 — Accessibility audit on a data UI

Produce a `Data-heavy admin` screen (CLI HTML/prototype tolerated if you declare tooling):

1. Inspect heading order + landmark usage.
2. Verify sortable columns expose **meaningful announcements** OR explain why visuals alone fail WCAG-aligned expectations.
3. Run automated scan; classify each finding Severity vs Noise.

Deliverable includes **fixes** applied for at least two real issues discovered.

Lab 6 — Cross-team contract incident

Fabricate changelog: `"orders.tax_cents"` nullable after migration; API neglected to update Zod/schema.

Produce **RCA**:

- Detection layer (monitoring/alerts/logs).
- TypeScript vs runtime validation interplay.
- User-visible symptom timeline.

Tie remediation to **backward-compatible API version** discipline.
